
PRODUCT REQUIREMENTS DOCUMENT

Loopy

Social-Commerce Thrift Marketplace for Instagram Sellers (India)

Table of Contents

1. Executive Summary
2. Problem Statement & Background
3. Competitive Landscape
4. Vision, Goals & Non-Goals
5. Success Metrics
6. Target Users & Personas
7. User Stories & Jobs-to-be-Done
8. Scope
9. System Architecture
10. Technology Stack & Framework Selection
11. Frontend Architecture
12. Backend Modules (Detailed)
13. Data Model (PostgreSQL)
14. API Design
15. Key Workflows
16. Security, Privacy & Compliance
17. Analytics & Instrumentation
18. Non-Functional Requirements
19. Scalability & Future Roadmap
20. Risks & Mitigations
21. Release Plan (MVP, 3–4 weeks)
22. Strategic Recommendations (Direct Pushback)
23. Open Questions
- Appendix A — Glossary
- Appendix B — Source References

Product Requirements Document — Loopy

Social-Commerce Thrift Marketplace for Instagram Sellers (India)

1. Executive Summary

Loopy is a social-commerce platform that gives Instagram thrift-clothing sellers in India their own independent online store (e.g. `loopy.com/{sellername}`) with built-in payments, escrow-style buyer protection, integrated logistics, seller KYC, ratings, and lightweight fraud scoring. It is positioned as "Shopify for Instagram thrift sellers," but differentiated from existing tools by **trust infrastructure**: a payment hold that protects buyers, verified sellers, and managed shipping with returns.

The MVP targets roughly **15 sellers and 100–500 orders/month**, with the architecture deliberately designed to scale to **thousands of orders/month within a year** and a full multi-seller marketplace thereafter. Revenue in the MVP comes from a **~5% commission** added at checkout; subscriptions and value-added services are deferred.

The MVP is scoped for a **3–4 week** initial build covering store creation, product management, cart/checkout, prepaid payments with escrow hold, manual seller payouts, and Shiprocket-based shipping with returns. COD, marketplace-wide search, recommendations, and advanced analytics are explicitly out of MVP scope.

2. Problem Statement & Background

Thrift sellers on Instagram run their businesses inside DMs and comment threads. This creates structural problems for both sides of the transaction:

For **sellers**, order management is manual and error-prone (screenshots, UPI IDs shared in DMs, spreadsheets for inventory). There is no professional storefront, no structured catalog, no automated payment reconciliation, and no logistics workflow. Drops sell out in minutes and overselling the same one-of-one item to two buyers is common.

For **buyers**, the experience is low-trust. Payment is typically a direct UPI transfer to a stranger with no recourse if the item never ships or arrives "not as described." There is no standardized return path, no verified-seller signal, and no neutral party holding funds until delivery.

Existing tools partially address the storefront problem (see Section 3) but largely leave the **trust and logistics gap** open. Loopy's wedge is to close that gap: a real storefront *plus* escrow-style protection *plus* managed shipping, so that a first-time buyer can transact with an unknown thrift seller with confidence.

3. Competitive Landscape

Platform	Commission	Seller payout	Key characteristics
DM2Buy	0%	Next bank day (via Cashfree)	Instagram-focused, free, unlimited listings, order management, basic themes, no-code, COD on Pro, custom domain (paid)
ZThrifs / Jettrips	Not public (likely higher)	Not disclosed	Curated vintage/thrift, quality-checked items, verified sellers
Obizee, Mallplaza, etc.	1–5%+ (varies)	Varies	Multichannel (Instagram, web, WhatsApp), custom stores, sometimes subscription / pay-to-list
Instagram Shopping	Platform-free	N/A	No mature Indian payment/checkout integration; checkout via DM in practice
Loopy (ours)	~5% (added to buyer price)	Hold ~48h, manual payout	Seller-owned stores, integrated escrow, managed logistics, fraud scoring, seller KYC, ratings, Shopify-like UX

Positioning. Loopy blends DM2Buy's frictionless onboarding with Jettrips-style trust infrastructure (verified sellers, quality signals) and adds an escrow-style hold plus integrated logistics. The competitive bet is that **trust and convenience are worth a modest fee** to both buyers (protection) and sellers (less manual work, fewer disputes).

Honest read of the landscape (see also Section 22). DM2Buy's 0% commission and next-day payout set a hard anchor on seller expectations: fast money, no fees. A 5% buyer-side surcharge and a 48h payout hold are both *worse* than the incumbent on the dimensions sellers care most about. Loopy must make the trust/logistics value unmistakable, or the commission and hold will be the reason sellers don't switch.

4. Vision, Goals & Non-Goals

4.1 Vision

Become the default commerce and trust layer for India's social-commerce long tail — starting with Instagram thrift fashion — so that any seller can run a professional, trusted store in minutes and any buyer can transact safely with a stranger.

4.2 Goals (MVP)

1. Let a seller go from sign-up to a live, shareable storefront with products in under a day (excluding KYC approval time).
2. Process a prepaid order end-to-end: checkout → payment → seller acceptance → shipment → delivery → payout, with no manual intervention in the happy path except payout approval.
3. Establish buyer trust through escrow-style hold, verified-seller badges, ratings, and a clear return path.
4. Keep unit economics viable at ~5% commission with no subscription.
5. Ship a stable MVP that 15 pilot sellers will use for real orders.

4.3 Non-Goals (MVP)

- Not a public marketplace with cross-seller search/discovery (each store is independent; discovery comes from the seller's own Instagram audience).
- Not COD-enabled.
- Not multi-seller carts / split shipments.

- Not an ML-heavy fraud or recommendation system.
- Not a custom-domain-per-seller product (path-based stores only in MVP).
- Not a native mobile app (mobile-first PWA instead).

5. Success Metrics

Category	Metric	MVP target	Why it matters
Adoption	Onboarded, KYC-approved sellers	15	Validates supply-side willingness to switch
Activation	Sellers with ≥ 1 live product within 7 days	$\geq 80\%$ of onboarded	Onboarding friction check
Volume	Orders / month	100–500	Core demand signal
Conversion	Checkout-start $\rightarrow$ paid conversion	$\geq 60\%$	Detects payment/UX friction (esp. no-COD impact)
Trust	Dispute rate (disputes / delivered orders)	$< 5\%$	Validates the trust thesis
Reliability	Payment success rate	$\geq 97\%$	PG integration health
Reliability	Order fulfillment SLA (accept within 24h)	$\geq 90\%$	Seller responsiveness
Finance	Payout accuracy (correct amount, on time)	100%	Trust + compliance critical
Performance	P95 API latency (key reads)	< 300 ms	UX
Availability	Uptime (checkout path)	99.9%	Revenue path

Instrumentation requirements for these metrics are specified in Section 17.

6. Target Users & Personas

Persona A — "Riya," the thrift seller (primary). 19–28, runs a thrift Instagram with 2k–50k followers, sells one-of-one curated pieces in drops. Phone-first, non-technical, time-poor during drops. Pain: managing DMs, payments, and shipping manually; overselling; chasing UPI confirmations. Needs: fast catalog upload, a link to share in bio/stories, automatic order capture, painless shipping labels, reliable payouts.

Persona B — "Aman," the buyer (primary). 18–30, discovers items via Instagram, impulse-driven, mobile-only, wary of paying strangers. Pain: fear of being scammed, no returns, no recourse. Needs: a fast checkout, confidence the seller is verified, protection if the item never arrives or is misdescribed.

Persona C — "Ops/Admin" (internal). Reviews KYC, resolves disputes, approves payouts, monitors fraud flags. Needs: a clear queue-based admin console and an audit trail.

7. User Stories & Jobs-to-be-Done

Seller: - As a seller, I want to create my store and upload products quickly so I can share a link during a drop. - As a seller, I want orders to appear automatically with buyer and payment status so I stop managing DMs. - As

a seller, I want to accept/reject an order and generate a shipping label in a couple of taps. - As a seller, I want to see my wallet balance and request a payout.

Buyer: - As a buyer, I want to browse a seller's store and pay securely in one flow. - As a buyer, I want to know the seller is verified and that my money is protected until delivery. - As a buyer, I want to track my order and raise a dispute with photo evidence if something is wrong. - As a buyer, I want to rate the seller and product after delivery.

Admin: - As an admin, I want to review and approve/reject seller KYC. - As an admin, I want to resolve disputes by refunding the buyer or releasing funds to the seller. - As an admin, I want to see flagged orders and platform-level metrics.

8. Scope

8.1 In scope (MVP — P0/P1)

Store creation and seller onboarding with KYC; product CRUD with image upload and inventory; public storefront and product pages; cart and prepaid checkout; Razorpay payment with HMAC-verified webhooks; escrow-style wallet hold (~48h); manual seller payouts via Razorpay Payouts; Shiprocket shipment creation, label, pickup, tracking, and returns; order lifecycle and status; buyer ratings; admin KYC approval and dispute resolution; OTP and Google login; rule-based fraud flags; transactional notifications (email + SMS/WhatsApp).

8.2 Out of scope (MVP — deferred to roadmap)

COD; marketplace-wide search and discovery; recommendations; multi-seller carts and split shipments; custom domain per seller; subscription tiers; in-app seller ↔ buyer chat; advanced BI dashboards; ML fraud/risk; native apps; automated payout scheduling.

8.3 Assumptions

- Each order contains items from exactly one seller.
- Most products are single-quantity (`quantity = 1`) one-of-one items.
- Sellers are onboarded manually and KYC-approved before they can transact.
- Buyers reach a store via a link the seller shares (Instagram bio/story/DM), not via on-platform discovery.
- Initial traffic is low (≤ 100 concurrent users, ≤ 10 orders/min).

8.4 Dependencies

- **Razorpay**: Payment Gateway (orders, capture, refunds) and Payouts (Contacts, Fund Accounts, payouts). Requires a Payouts-enabled account and IP allowlisting.
- **Shiprocket**: Order create/ship/pickup/label/track, returns (reverse logistics).
- **OTP/SMS/WhatsApp gateway** (e.g. Twilio / WhatsApp Business API) and **email** (SES/Mailgun).
- **Cloud**: AWS or GCP (compute, managed Postgres, managed Redis, object storage, CDN).
- **Regulatory**: RBI Payment Aggregator rules govern how buyer funds may be held; see Sections 16 and 22 — this is a hard dependency on getting the money-flow model right.

9. System Architecture

9.1 Architectural style

A **modular monolith**: a single deployable codebase organized into clearly separated domain modules, with internal boundaries drawn so that high-load modules (Payments, Orders) can later be extracted into independent services. This is the right call for a 2-dev + 1-DevOps team: faster to build, simpler to deploy and reason about, and cheaper to operate than microservices, while keeping a clean path to decomposition.

9.2 High-level component view

```

Front-end (Next.js) <--> API (NestJS HTTP server) <--> PostgreSQL
                        |
                        Redis (cache / locks / queues)
                        |
                        External APIs (Razorpay, Shiprocket, OTP/Email)
                        |
                        Object storage (S3) + CDN (CloudFront)
  
```

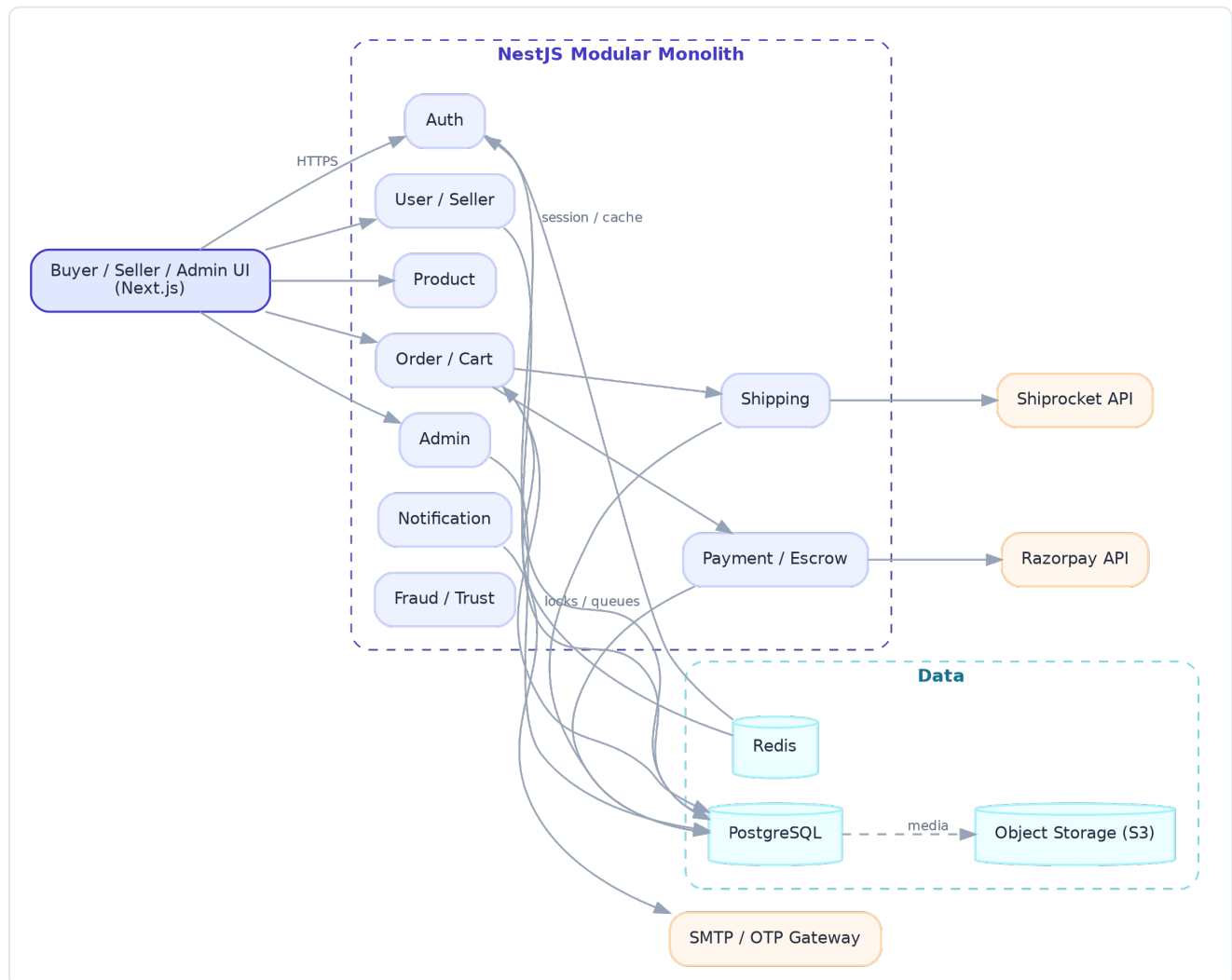


Figure 1 — High-level system architecture (modular monolith)

9.3 Modules

Module	Responsibility
Auth	OTP login (phone/email), Google OAuth, JWT issue/refresh
User / Seller	Profiles, roles, KYC docs, store settings
Product / Store	Catalog CRUD, inventory, image processing
Order / Cart	Cart, order creation, status transitions
Payment / Escrow	Razorpay payments, escrow hold, wallet, payouts
Shipping	Shippo create/ship/pickup/track, returns
Notification	Email / SMS / WhatsApp / push, templates, retries
Admin	Seller approvals, dispute resolution, reports
Fraud / Trust	Rule-based scoring, event logging, trust scores

9.4 Event-driven internals

Asynchronous flows are handled via an in-process event bus backed by Redis queues (BullMQ). Example: payment capture emits `OrderPaid`; consumers credit the seller wallet, schedule the payout-eligibility timer, and trigger notifications. This decouples slow/external work from the request path and gives a clean seam for later service extraction.

Key events: `OrderCreated`, `OrderPaid`, `OrderAccepted`, `OrderRejected`, `ShipmentCreated`, `OrderDelivered`, `DisputeOpened`, `PayoutEligible`, `PayoutCompleted`, `RefundIssued`.

9.5 Cross-cutting platform concerns

- **Stateless app containers** (sessions/locks in Redis) for horizontal scaling.
- **Caching** of hot reads (categories, active product lists) in Redis.
- **Media** via S3 + CloudFront with signed URLs; Next.js image optimization.
- **Rate limiting / throttling** (NestJS Throttler) on all public endpoints, strict on auth.
- **API versioning** (`/api/v1/...`) from day one.

10. Technology Stack & Framework Selection

This section consolidates every framework, library, and service, with rationale and the decision recorded. Items marked (*recommended addition*) are my proposals beyond the original brief.

10.1 Frontend

Concern	Choice	Rationale / notes
Framework	Next.js (App Router) + TypeScript	SSR/ISR for crawlable storefronts; one codebase for storefront + dashboard
Styling	Tailwind CSS	Fast, consistent, mobile-first
UI components	Headless UI / shadcn-ui (<i>recommended</i>)	Accessible primitives without heavy theming lock-in
State	Zustand (cart) + React Context (auth)	Minimal global state; avoid Redux overhead
Forms	React Hook Form + Zod (<i>Zod recommended over Yup</i>)	Type-safe schemas shared conceptually with backend DTOs
Data fetching	TanStack Query (<i>recommended</i>)	Caching, retries, and request dedup for dashboard reads
HTTP	Axios wrapper with JWT interceptor	Centralized auth + error handling
PWA	next-pwa	Installable, offline-friendly storefront (P1)
SEO	next/head + JSON-LD product schema	Storefront discoverability
Testing	Vitest + React Testing Library + Playwright (<i>e2e, recommended</i>)	Component + critical-flow coverage

10.2 Backend

Concern	Choice	Rationale / notes
Runtime / framework	Node.js + NestJS (TypeScript)	Modular DI architecture; controller/service/repository pattern
ORM	Prisma (<i>recommended over raw/TypeORM</i>)	Type-safe schema + migrations; good DX for a small team
Validation	class-validator + DTOs / Zod	Strict input validation, global pipes
Auth	Passport (JWT + Google strategies), NestJS Guards	Access (~15 min) + refresh (~7 d) tokens
Queues / jobs	BullMQ (Redis)	Payouts, notifications, fraud cron, webhook retries
Scheduling	@nestjs/schedule + Bull repeatable jobs	Payout-eligibility timer, nightly fraud scoring
Logging	Pino / Winston	Structured logs with request ID, user ID
API docs	Swagger (@nestjs/swagger) (<i>recommended</i>)	Living API contract for the team
Payments	Razorpay Node SDK	Orders, capture, refunds, payouts
Shipping	Shippo REST (typed client)	Create/ship/pickup/track/returns
Notifications	SES/Mailgun (email), Twilio / WhatsApp Business (SMS/WA)	Transactional messaging
Testing	Jest (unit) + Supertest (e2e)	Critical services and routes

10.3 Data & infrastructure

Concern	Choice	Rationale / notes
Primary DB	PostgreSQL (Amazon RDS, Multi-AZ)	Relational integrity for money/orders
Cache / locks / queues	Redis (ElastiCache)	Sessions, hot cache, distributed locks, BullMQ
Object storage	S3	Product images, KYC docs (private, signed URLs)
CDN	CloudFront	Static assets + image delivery
Containers	Docker (multi-stage builds)	Reproducible builds
Orchestration	ECS/Fargate (start) → EKS (later)	Managed containers; minimal ops at MVP
IaC	Terraform	Reproducible infra
CI/CD	GitHub Actions	Build/test/deploy to staging + prod
Secrets	AWS Secrets Manager	JWT/Razorpay/Shiprocket/DB creds
Monitoring	CloudWatch + Sentry (<i>recommended</i>) + Prometheus metrics	Errors, latency, business metrics
Error tracking	Sentry (<i>recommended</i>)	Frontend + backend exception capture

10.4 Cloud note

The original brief allows AWS or GCP. Given existing Google for Startups Cloud credits, **GCP is worth a serious cost evaluation** (Cloud Run instead of Fargate, Cloud SQL for Postgres, Memorystore for Redis, GCS + Cloud CDN). The architecture is cloud-agnostic by design (containers + managed Postgres/Redis/object storage), so the decision can be driven by credit runway rather than lock-in. Pick one and standardize; do not split services across both.

11. Frontend Architecture

Two surfaces in one Next.js codebase:

- **Storefront (buyer-facing, public):** seller store home, product page, cart, checkout, order confirmation, tracking, reviews, buyer profile.
- **Dashboard (seller + admin, authenticated):** product/order/payout management; admin approvals and disputes.

Routing (file-based): - `/s/[sellerUsername]` — store home (SSR/ISR for SEO) - `/s/[sellerUsername]/p/[productSlug]` — product page (ISR with on-demand revalidation; SSR for live stock on the critical path) - `/cart`, `/checkout`, `/orders/[id]` — buyer flows - `/seller/dashboard`, `/seller/products`, `/seller/orders`, `/seller/wallet` — protected seller routes - `/admin/...` — protected admin console

Rendering strategy. Storefront pages use ISR for crawlability and speed, with SSR fallback for checkout where live inventory matters. Dashboard is auth-gated and can be CSR-heavy.

Auth on the client. JWT in HttpOnly cookie (preferred over localStorage for XSS resistance); refresh handled transparently by the Axios interceptor; route guards on all `/seller` and `/admin` paths.

Quality. React error boundaries on critical components; explicit empty/loading/error states; accessibility (contrast, alt text, labelled inputs); toast feedback on success/failure.

Suggested folder structure:

```

/app                # routes (storefront + dashboard + admin)
/components         # Navbar, ProductCard, OrderTimeline, etc.
/lib               # API client, auth helpers
/stores            # Zustand stores (cart)
/contexts          # AuthContext
/hooks            # custom hooks
/styles            # tailwind config, globals

```

12. Backend Modules (Detailed)

Pattern: controller → service → repository, DTOs validated by class-validator, guards for auth/RBAC, global exception filter.

Auth. `/auth/login`, `/auth/verify-otp`, `/auth/google`, `/auth/refresh`. OTP stored in Redis with short TTL; block disposable emails; JWT access ~15 min + hashed refresh tokens in DB.

User / Seller. `/users/me`, `/sellers/me`, `/sellers/register`, `/sellers/kyc`. On registration, create seller with `status = pending`; KYC docs (PAN/Aadhaar/bank) uploaded via signed S3 URLs; admin approves; validate PAN format, Aadhaar length, IFSC (ideally via Razorpay bank validation).

Product. `/products` CRUD. Fields: title, description, price, condition, size, brand, category, images[], quantity. Thumbnail generation on upload. Inventory decrement guarded by DB row locks / Redis locks to prevent oversell of one-of-one items.

Order / Cart. `/cart`, `/orders/checkout`, `/orders/:id`. Checkout reserves inventory, creates a Razorpay order, and on verified payment marks the order `Paid` and emits `OrderPaid`. Order status enum: `PendingPayment` → `Paid` → `Accepted` → `Shipped` → `Delivered` → `Completed`, with `Cancelled` / `Disputed` / `Refunded` branches.

Payment / Escrow. Webhooks (`/payment/webhook`) and payout request endpoints. On payment success, credit seller wallet (minus commission) without transferring to bank; release after the dispute window via Razorpay Payouts; refunds + wallet adjustment on dispute. Idempotency keys on payouts. Ledger maintained in `wallets` and `payouts`.

Shipping. `/shipments` + Shippo webhooks. After acceptance: `order_create` → `order_ship` (AWB) → `order_pickup_schedule` → label; track via webhook/poll; manual AWB entry fallback; reverse shipment for returns.

Notification. Channels: email (SES/Mailgun), SMS/WhatsApp (Twilio/WA Business), push (optional). Triggered on each lifecycle event; templated; retried via queue.

Admin. `/admin/sellers`, `/admin/disputes`, `/admin/reports`. KYC approve/reject, dispute resolution (refund buyer / release to seller), platform analytics.

Fraud / Trust. Rule-based scoring (see Section 15.4), event logging, nightly cron, admin review queue. Modularized so rules can be swapped for ML later.

NestJS practices. DTOs + class-validator; Passport guards; `/v1` versioning; global pipes/filters; CORS for the frontend origin + IP allowlist for payout APIs; structured logging; BullMQ for background jobs.

13. Data Model (PostgreSQL)

Normalized relational schema. Key tables:

Table	Key columns	Notes
users	id, name, email, phone, password_hash, role (buyer/seller/admin), timestamps	Email/phone unique; role enum
sellers	id (FK users), store_name, username (unique), description, banner_url, logo_url, kyc_status, contact_id (Razorpay), fund_account_id, timestamps	1:1 with users; kyc_status: pending/approved/rejected
products	id, seller_id, title, description, price, condition, size, brand, category_id, images (jsonb), quantity, is_active, timestamps	Inventory via quantity
categories	id, name	Optional in MVP
carts	id, user_id, timestamps	One cart per buyer/session
cart_items	id, cart_id, product_id, quantity	
orders	id, buyer_id, seller_id, total_amount, commission_amount, shipping_charge, status, payment_id, timestamps	Status enum (see Section 12)
order_items	id, order_id, product_id, unit_price, quantity, subtotal	Price locked at order time
payments	id, order_id, razorpay_order_id, razorpay_payment_id, method, amount, status, paid_at	
wallets	id, seller_id, balance	Escrow-held seller balance
payouts	id, seller_id, amount, status, request_date, paid_date, remarks	
shipments	id, order_id, courier_name, awb_number, pickup_date, delivery_date, status	From Shiprocket
reviews	id, order_id, product_id, rating (1–5), comment, images, created_at	Post-delivery
disputes	id, order_id, buyer_id, seller_id, issue_type, description, status, created_at, resolved_at	Admin-resolved
sessions	id, user_id, refresh_token (hashed), expires_at	Refresh-token management
audit_logs (recommended)	id, actor_id, action, entity, entity_id, metadata (jsonb), created_at	Financial/admin action trail

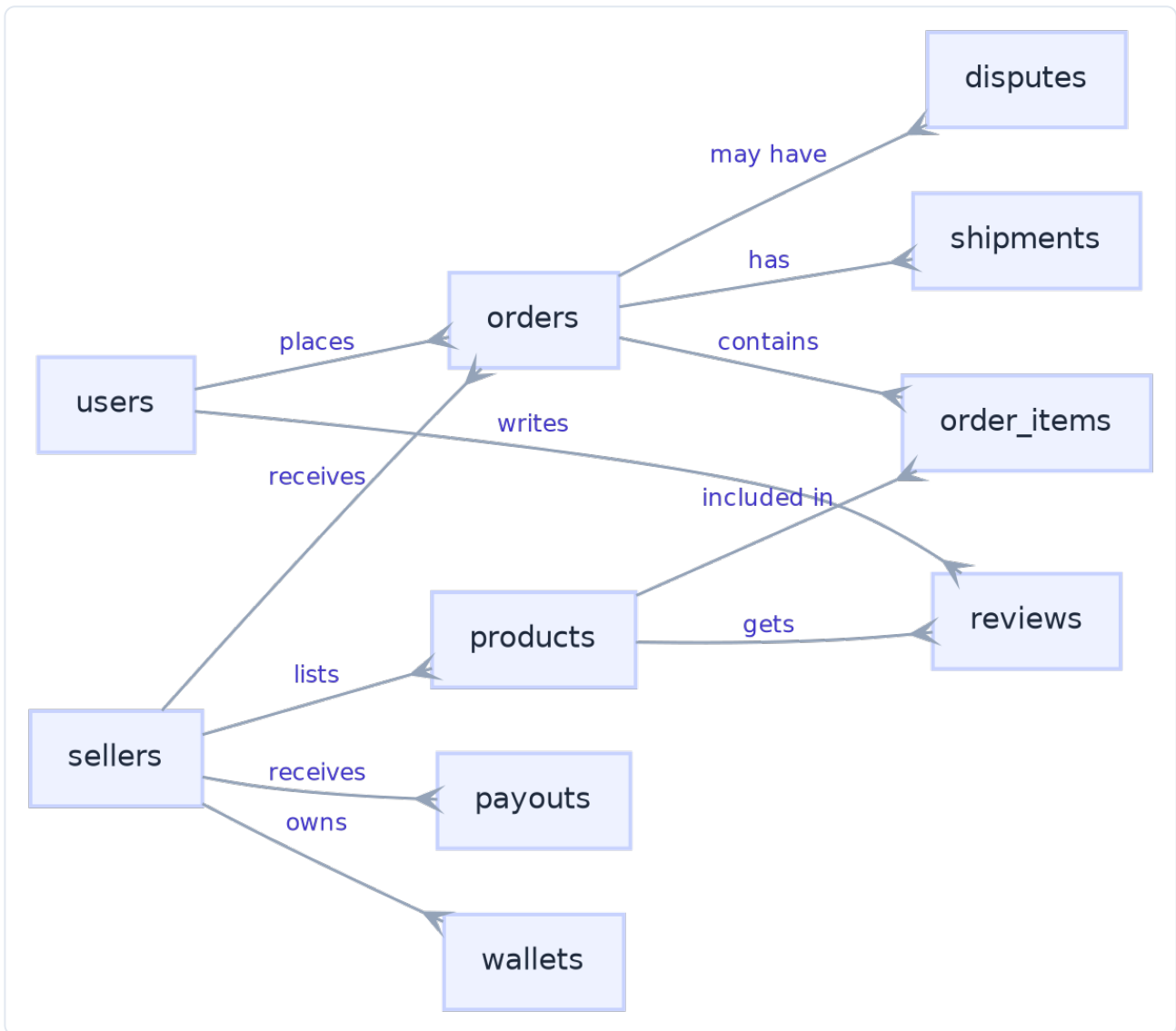


Figure 2 — Entity relationships (PostgreSQL data model)

Constraints & integrity. Unique on `users.email`, `users.phone`, `sellers.username`. Indexes on `products.seller_id`, `orders.seller_id`, `orders.buyer_id`, `payments.razorpay_order_id`. Multi-step flows wrapped in transactions at `REPEATABLE READ`. Inventory uses `SELECT ... FOR UPDATE` (or Redis lock) at checkout. KYC PII (Aadhaar, account numbers) encrypted/hashed at rest; never stored in clear text. Data retention: archive/partition orders by date as volume grows.

14. API Design

RESTful JSON under `/api/v1`, Bearer JWT on protected routes. Representative endpoints:

Resource	Method	Endpoint	Auth	Notes
Auth	POST	/auth/login	public	Sends OTP
Auth	POST	/auth/verify	public	Returns access + refresh JWT
Auth	POST	/auth/refresh	refresh JWT	New access token
Seller	POST	/sellers/register	public	Creates pending seller
Seller	POST	/sellers/kyc	seller	Multipart KYC upload
Seller	GET	/sellers/me	seller	Own profile
Products	GET	/sellers/{sid}/products	public	Seller catalog
Products	POST	/sellers/me/products	seller	Create product
Products	PUT	/sellers/me/products/{pid}	seller (owner)	Edit product
Cart	GET	/cart	buyer	Current cart
Cart	POST	/cart/items	buyer	Add item (stock-checked)
Orders	POST	/orders/checkout	buyer	Creates order + Razorpay order
Orders	POST	/orders/confirm	buyer	Verifies signature, marks Paid
Orders	GET	/orders/{id}	role-based	Order detail
Orders	POST	/orders/{id}/accept	seller	Accept order
Orders	POST	/orders/{id}/ship	seller	Create shipment, return AWB
Orders	POST	/orders/{id}/delivered	webhook/admin	Mark delivered
Reviews	POST	/orders/{id}/reviews	buyer	Post-delivery review
Payouts	GET	/seller/wallet	seller	Balance/pending/paid
Payouts	POST	/seller/payouts	seller	Request payout
Admin	GET	/admin/sellers	admin	Pending KYC
Admin	POST	/admin/sellers/{sid}/approve	admin	Approve/reject
Webhooks	POST	/payment/webhook	Razorpay signature	Capture/refund/transfer
Webhooks	POST	/shipment/webhook	Shiprocket	Tracking updates

Standards. HTTPS only; strict input validation (e.g. `price > 0`); standard status codes (400/401/403/404/409/422/500); JSON error bodies; **idempotency** on checkout and on both webhooks (detect duplicate `payment_id` / tracking event); IP allowlisting for payout calls.

15. Key Workflows

15.1 Payment & escrow flow

1. Buyer pays → backend creates a Razorpay order (amount = items + shipping + commission).
2. Buyer completes payment via Razorpay Checkout (UPI/card/netbanking).
3. Razorpay webhook → backend verifies HMAC `X-Razorpay-Signature` → marks order `Paid`.

4. Seller wallet credited (minus commission); funds **held**, not transferred (escrow-style).
5. Seller accepts within ~24h; on reject/timeout, buyer is refunded and inventory released.
6. Seller ships via Shiprocket.
7. On delivery, a **48h dispute window** opens.
8. No dispute after 48h → Razorpay Payout to seller's Fund Account.
9. Payout logged; in MVP, payout is manually approved by admin/ops.

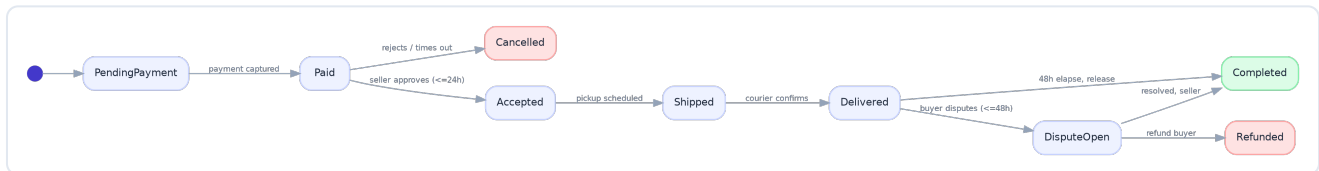


Figure 3 — Order / escrow lifecycle state machine

Razorpay specifics. On KYC approval, create Contact + Fund Account, store IDs. Use `POST /v1/orders`, webhook `payment.captured`, `POST /v1/payouts`, `POST /v1/payments/:id/refund`. Use Idempotency-Key on payouts. Handle payout failures (bank rejection) with logging and alerts.

Compliance flag. The "platform holds funds in its own account and pays out manually" model has real RBI Payment Aggregator implications and should not ship as literally described. See Section 16.2 and Section 22.1 for the corrected approach (use Razorpay Route / managed escrow rather than self-custody of funds).

15.2 Shipping flow

Seller clicks "Ship" → `order_create` (buyer address, weight, COD=0) → `order_ship` (AWB) → `order_pickup_schedule` → `generate_shipment_label`. Track via webhook/poll; mark delivered on `Delivered`. Manual AWB entry as fallback. Validate serviceable pincodes before order creation. Idempotent webhook handling.

15.3 Returns flow

Buyer requests return (within the stated return window) → seller/admin approves → reverse Shiprocket shipment scheduled → on confirmed pickup, refund buyer and adjust seller wallet. Multi-item / split returns are out of MVP scope.

15.4 Fraud & trust (rule-based, MVP)

Hidden seller trust score (signup age, profile completeness, KYC, dispute ratio, ratings); buyer risk checks (new account + large order, address mismatch); order velocity caps (e.g. 3/day); India-only seller IP check; optional image-duplicate check; dispute requires photo evidence; admin review queue for flags; nightly scoring cron; all triggered rules logged. Modular so ML can replace rules later.

16. Security, Privacy & Compliance

16.1 Security

JWT access (~15 min) + hashed refresh (~7 d); RBAC guards (buyer/seller/admin) on every protected route; secrets in AWS Secrets Manager (never in code); KYC docs and images in private S3 with signed URLs; PCI handled by offloading all card entry to Razorpay (card data never touches our servers); CSP/HSTS headers;

per-IP and per-user rate limits (strict on OTP/login); parameterized queries via ORM; class-validator on all inputs; OWASP Top 10 mitigation (SQLi, XSS, CSRF).

16.2 Compliance (India)

- **RBI Payment Aggregator rules** govern holding and settling buyer funds. The platform must **not** custody buyer money in its own account beyond what the PA framework allows. Use a licensed partner's escrow/route capability (Razorpay Route) so settlement-after-delivery is implemented within a compliant structure rather than by self-holding funds. Treat this as gating for the money flow, not an afterthought.
 - **KYC**: collect and verify PAN, Aadhaar, bank/IFSC; sellers cannot transact until approved.
 - **Data protection**: encrypt/hash sensitive PII at rest; minimize what is stored; align with the DPDP Act (consent, purpose limitation, breach handling) — relevant given the founder's broader compliance positioning.
 - **Audit**: log all financial transactions (payouts, refunds) with timestamps and actor.
-

17. Analytics & Instrumentation

To make Section 5 measurable, instrument from day one:

- **Event tracking** (server-side preferred for money events): `signup`, `kyc_submitted`, `kyc_approved`, `product_created`, `store_link_opened`, `checkout_started`, `payment_succeeded`, `payment_failed`, `order_accepted`, `order_shipped`, `order_delivered`, `dispute_opened`, `payout_requested`, `payout_completed`, `review_submitted`.
 - **Funnels**: store visit → checkout start → paid; signup → KYC approved → first product → first sale.
 - **Dashboards**: GMV, orders/day, payment success rate, dispute rate, payout accuracy/timeliness, P95 latency, uptime.
 - **Tooling**: product analytics (PostHog (*recommended*), self-hostable, privacy-friendly), backend metrics via Prometheus, errors via Sentry, infra via CloudWatch.
-

18. Non-Functional Requirements

ID	Requirement	Target
NFR-1	API latency (key reads, P95)	< 300 ms
NFR-2	Throughput (MVP)	100 concurrent users, 10 orders/min
NFR-3	Availability (checkout path)	99.9%, multi-AZ
NFR-4	Payment success rate	≥ 97%
NFR-5	Data consistency	Transactional inventory/order integrity
NFR-6	Security	OWASP Top 10 mitigated; PCI offloaded to Razorpay
NFR-7	Auditability	All financial events logged with timestamp + actor
NFR-8	Rate limits	~1000 req/min/IP; 5 failed logins/hour/user
NFR-9	Observability	Metrics endpoint (error rate, order rate, payment success)
NFR-10	Extensibility	New modules (e.g. recommendations) addable without rewrite
NFR-11	Backups	Daily Postgres snapshots; tested restore
NFR-12	Recoverability	Incomplete transactions roll back cleanly

19. Scalability & Future Roadmap

The MVP is built lean but along clean seams so the same codebase scales without a rewrite. This section is the forward-looking view requested for the build.

19.1 Scaling the MVP architecture (to thousands of orders/month)

- **Horizontal app scaling:** stateless containers behind a load balancer; autoscale on CPU/request volume.
- **Database:** scale RDS vertically first; add read replicas for storefront reads; introduce connection pooling (PgBouncer). Partition `orders` / `payments` by month as volume grows.
- **Caching:** expand Redis caching of storefront product lists and seller pages; cache invalidation on product update via events.
- **Media:** all images through CloudFront; pre-generate responsive sizes asynchronously.
- **Queues:** move heavy/external work (shipping calls, notifications, fraud scoring) fully onto BullMQ workers that scale independently of the API.
- **Search:** when on-platform discovery arrives, introduce a dedicated search engine (OpenSearch / Meilisearch / Typesense) rather than querying Postgres with `LIKE`.

19.2 Service extraction (when load justifies it)

The modular monolith was chosen precisely so that the **Payment/Escrow** and **Order** modules can be extracted into standalone services first (they carry the highest load and the strictest reliability/compliance requirements), communicating over the existing event bus (graduate Redis/BullMQ to a managed broker like SQS/SNS or Kafka). Extract only when a module's scaling or deployment cadence genuinely diverges — not preemptively.

19.3 Product roadmap (phased)

Phase 2 — Marketplace & discovery. Cross-seller search, categories, curated drops, multi-seller carts and split shipments, custom domains per seller, COD (with risk controls).

Phase 3 — Monetization depth. Subscription tiers for power sellers, promoted listings, value-added logistics, payment options beyond prepaid.

Phase 4 — Intelligence. ML-based fraud/risk replacing rule engine; recommendations and personalization; demand forecasting for sellers; image-based catalog assistance.

Phase 5 — Platform. Seller ↔ buyer chat, loyalty, broader social-commerce verticals beyond thrift fashion, deeper analytics/BI for sellers.

19.4 Engineering enablers to add as you scale

API gateway in front of extracted services; feature flags for safe rollout; blue-green/canary deploys; distributed tracing (OpenTelemetry); a proper data warehouse + dbt for analytics once event volume is meaningful.

20. Risks & Mitigations

Risk	Impact	Likelihood	Mitigation
RBI PA/escrow non-compliance from self-custody of funds	Critical (legal/shutdown)	Medium	Use Razorpay Route / licensed escrow; do not hold funds directly; legal review before launch
5% buyer surcharge + 48h hold loses sellers to 0%-fee incumbents	High	High	Lead with trust/logistics value; test seller-side vs buyer-side fee; consider lower intro commission
No COD suppresses conversion in price-sensitive segments	High	Medium	Measure checkout conversion; plan controlled COD in Phase 2
3–4 week MVP timeline vs scope with 2 devs	High	High	Cut to true P0; stub manual fallbacks; sequence per Section 21
Overselling one-of-one items (race condition)	High	Medium	DB row locks / Redis locks + transactional decrement
Webhook loss/duplication (payment, shipping)	High	Medium	Idempotency keys, non-200 to force retry, full callback logging
Payout errors / double payouts	Critical (trust + money)	Low	Idempotency keys, ledger reconciliation, manual approval in MVP
Seller unresponsiveness	Medium	Medium	Auto-cancel/refund at 24h; escalate to admin
KYC/PII breach	Critical	Low	Encryption at rest, private S3, least-privilege access, DPDP alignment
Single-cloud cost overrun	Medium	Medium	Use available Google credits; right-size; autoscale floors low

21. Release Plan (MVP, 3–4 weeks)

Priorities: **must-have** = store creation, product CRUD, cart/checkout, payment, payouts, shipping. **Deferred** = search, chat, recommendations, COD.

Week 1 — Foundation. Repo, CI/CD, Docker. Auth (OTP) + seller registration with KYC upload. Postgres schema (users, sellers) + migrations. Frontend skeleton: landing, login, seller signup.

Week 2 — Store & products. Seller dashboard skeleton. Product module (API + DB + forms, S3 image upload). Public storefront list + detail. Cart (add/view, session-based).

Week 3 — Orders & payments. Checkout (Razorpay order creation + payment page). Payment webhook → finalize order. Confirmation page. Wallet credit on payment. Seller order list with Accept/Reject (+ refund on reject).

Week 4 — Logistics & finalize. Shiprocket integration (shipment on acceptance, AWB). Buyer tracking page. Admin: pending-seller list + KYC approval. Notification emails (receipt, shipping). QA, responsive polish, loading states, production deploy.

Demo to founders weekly; keep mobile usability paramount; merge and test continuously.

Realistic note. This timeline is aggressive for the full scope with two developers. If something slips, protect the money path (checkout → payment → payout) and shipping; defer admin polish, reviews, and fraud rules to a fast-follow. Treat the compliance/money-flow decision (Section 16.2) as a Week-0 blocker, not a Week-4 detail.

22. Strategic Recommendations (Direct Pushback)

These are my own opinions, separated from the spec so the team can accept or reject them deliberately.

22.1 Fix the money-flow model first. As written, the platform receives buyer funds, holds them ~48h in its own wallet, and pays sellers manually. That is functionally payment aggregation with self-custody of funds, which is squarely in RBI PA territory. Implement settlement-after-delivery through Razorpay Route (or an equivalent licensed escrow/marketplace settlement product) so funds are held within a compliant structure and "release after 48h" is a settlement instruction, not you holding cash. Get this reviewed by someone who knows PA rules before writing payout code.

22.2 Reconsider who pays the 5%. Adding 5% to the buyer's price competes directly against a 0%-commission incumbent on the most visible number a buyer sees. Two alternatives worth A/B testing with pilot sellers: (a) take the commission from the seller's payout (invisible to the buyer, framed as "we handle payments, shipping, and protection for 5%"), or (b) a lower intro commission (2–3%) to win the first cohort, raised later once trust value is proven. The trust story justifies a fee; it may not justify a *visible buyer surcharge*.

22.3 No-COD is a real conversion risk. COD still drives a large share of Indian e-commerce, especially for first-time, price-sensitive buyers buying from unknown sellers — exactly Loopy's buyer. Prepaid-only is defensible for an MVP (simpler, lower fraud), but instrument checkout conversion carefully and treat controlled COD as an early Phase-2 item, not a distant one.

22.4 Tighten the trust narrative into the UI. The differentiator is trust, so make it impossible to miss: a persistent "Protected by Loopy — money held until you confirm delivery" badge at checkout, the verified-seller badge on every product card, and a one-line return promise above the fold. If trust is the reason to pay more, it has to be visible at the moment of payment.

22.5 Build the admin/ops console as a first-class surface, not an afterthought. With manual KYC approval, manual payouts, and manual dispute resolution, ops is the product in the MVP. A clean queue-based admin console with an audit trail will do more for early reliability than most buyer-side polish.

22.6 Pick the cloud by credit runway and commit. Given existing Google for Startups credits, price out GCP (Cloud Run + Cloud SQL + Memorystore + GCS/Cloud CDN) against AWS before standardizing. The stack is portable; the decision should be economic. Avoid splitting infrastructure across both clouds.

22.7 Reframe "Loopy" naming/URL early if it's a placeholder. The store URL pattern (`loopy.com/{sellername}`) and brand should be locked before SEO and seller links proliferate, since changing it later breaks shared links and crawled pages.

23. Open Questions

1. **Money flow:** confirmed Razorpay Route (or equivalent) for compliant settlement before any payout code is written?
 2. **Commission model:** buyer-side surcharge vs seller-side deduction vs lower intro rate — which do we pilot?
 3. **Return window:** the document references both a 48h dispute window and a 14-day return promise — what is the authoritative buyer-facing policy?
 4. **Payout cadence:** manual approval only in MVP, or auto-release after 48h with manual override?
 5. **Notifications:** SMS vs WhatsApp as the primary buyer channel at launch (cost vs deliverability)?
 6. **Seller chargeback liability:** does the platform cover disputed amounts for the first cohort, or is it always deducted from the seller wallet?
 7. **Store URL:** path-based (`/s/{seller}`) confirmed for MVP, with subdomains deferred?
 8. **Cloud:** AWS or GCP — decided against credit runway?
-

Appendix A — Glossary

- **Escrow-style hold:** funds held (within a compliant settlement structure) between payment and delivery confirmation before release to the seller.
- **AWB:** Air Waybill — the courier tracking number for a shipment.
- **KYC:** Know Your Customer — seller identity/bank verification (PAN, Aadhaar, bank/IFSC).
- **PA/PG:** Payment Aggregator / Payment Gateway — RBI-regulated categories governing how platforms handle funds.
- **ISR/SSR:** Incremental Static Regeneration / Server-Side Rendering — Next.js rendering strategies.
- **RBAC:** Role-Based Access Control.
- **DPDP:** Digital Personal Data Protection Act (India).
- **GMV:** Gross Merchandise Value.

Appendix B — Source References

Razorpay Payment Gateway & Payouts documentation; Shiprocket API documentation (`order_create` , `order_ship` , `order_pickup_schedule` , `generate_shipment_label` , tracking, returns); RBI Payment Aggregator guidelines; competitive references (DM2Buy, Jettrips). These informed the payment/escrow design, logistics flow, compliance posture, and MVP feature set.